

Mini-Jogo de Basquetebol 3D

Plano Completo de Melhorias — JIGS 2025/2026

Este documento consolida todas as melhorias planeadas e discutidas para o projeto, organizadas em três patches. Cada melhoria inclui descrição, ficheiros afetados, pseudocódigo e ordem de implementação recomendada.

Índice Geral de Melhorias

Patch	#	Melhoria	Ficheiros	Dificuldade
1	1.1	Corrigir teclas A/D invertidas	main.js	Baixa
1	1.2	Movimento estilo 2K (câmara fixa, jogador maior)	main.js, camera.js	Média
1	1.3	Rede do cesto invertida	court.js	Baixa
1	1.4	Bola e cesto maiores	constants.js	Baixa
1	1.5	Câmara limitada ao campo	camera.js	Baixa
1	1.6	Campo — reduzir brilho e ajustar escala visual	visual.js, camera.js	Baixa
2	2.1	Câmara vira ao cruzar o meio campo	camera.js, main.js	Média
2	2.2	Sistema de lançamento: precisão pura (estilo 2K)	shot.js, constants.js	Média
2	2.3	Novo cenário exterior — campo de parque	court.js, world/	Alta
2	2.4	Ciclo dia/noite com transição suave (teclas T)	main.js, world/, ui.js	Alta
3	3.1	Aro maior (exagerado para jogabilidade)	constants.js, scoring.js	Baixa
3	3.2	Animações procedurais do jogador	player.js	Alta
3	3.3	Céu procedural mais realista	world/sky.js (novo)	Média
3	3.4	Câmara em primeira pessoa com lock-on automático	camera.js, main.js	Alta

PATCH 1

Correções Base e Qualidade de Vida

Controlos, escala visual, rede e câmara — fundação do projeto

1.1 — Corrigir Teclas A/D Invertidas

Pressionar A move o jogador para a direita e D para a esquerda — o sinal do eixo X está trocado em `updatePlayerMovement` no `main.js`.

Código atual (errado):

```
const moveX = (input.isDown('KeyD') ? 1 : 0) - (input.isDown('KeyA') ? 1 : 0);
```

Código corrigido:

```
const moveX = (input.isDown('KeyA') ? 1 : 0) - (input.isDown('KeyD') ? 1 : 0);
```

1.2 — Movimento Estilo 2K (Opção A)

O sistema atual usa Q/E para rodar o jogador manualmente. O novo sistema remove Q/E e faz o jogador rodar automaticamente e suavemente para a direção do movimento. A câmara fica sempre atrás, fixa no mundo.

Comportamento	Descrição
WASD relativo à câmara	W = longe da câmara, S = perto, A = esquerda, D = direita
Rotação automática	<code>playerYaw</code> calculado com <code>atan2</code> da direção de movimento
Lerp suave	<code>lerpAngle(playerYaw, targetYaw, 1 - exp(-delta * 10))</code> para rotação fluída
Sem Q/E	Teclas Q e E removidas completamente do <code>InputController</code>

Pseudocódigo em `main.js`:

```
// Calcular direção de movimento relativa à câmara (yaw fixo = 0 pois câmara é fixa)
const moveDir = new THREE.Vector3(moveX, 0, -moveZ).normalize(); if (moveDir.length() > 0.01) {
  const targetYaw = Math.atan2(moveDir.x, moveDir.z); playerYaw =
  lerpAngle(playerYaw, targetYaw, 1 - Math.exp(-delta * 10)); }
```

1.3 — Rede do Cesto Invertida

O `ConeGeometry` do `Three.js` tem a base em baixo por defeito, colocando a abertura da rede para cima. Adicionar `rotation.x = Math.PI` vira o cone corretamente.

Adicionar em `court.js` após criar o `net mesh`:

```
net.rotation.x = Math.PI; // base do cone fica em cima – abertura vira para baixo
```

1.4 — Bola e Cesto Maiores

Os valores reais ficam demasiado pequenos no ecrã. Aumentar ~50% mantendo a proporção aro/bola para que a bola continue a entrar.

Constante	Atual	Novo	Ficheiro
<code>BALL.radius</code>	0.12	0.18	<code>constants.js</code>
<code>COURT.rimRadius</code>	0.225	0.34	<code>constants.js</code>

COURT.rimTubeRadius	0.02	0.03	constants.js
PHYSICS.rimSegmentRadius	0.02	0.03	constants.js

1.5 — Câmara Limitada ao Campo

A câmara pode sair para fora dos limites do campo. Adicionar clamp após o lerp de posição em camera.js com 1 metro de margem para não cortar a visão abruptamente.

Adicionar em camera.js após camera.position.lerp(...):

```
camera.position.x = THREE.MathUtils.clamp( camera.position.x, -COURT.halfWidth - 1,
COURT.halfWidth + 1 ); camera.position.z = THREE.MathUtils.clamp( camera.position.z,
-COURT.halfLength - 1, COURT.halfLength + 1 );
```

1.6 — Campo: Reduzir Brilho e Ajustar Escala Visual

Ajuste	Atual	Novo	Ficheiro
Floor roughness	0.44	0.82	court.js
Floor metalness	0.02	0.0	court.js
Câmara FOV	75°	65°	camera.js
followOffset Y	2.25	3.2	main.js (PLAYER)
followOffset Z	-3.8	-5.5	main.js (PLAYER)

PATCH 2

Cenário, Câmara e Sistema de Lançamento

Exterior, ciclo dia/noite, câmara inteligente e precisão pura

2.1 — Câmara Vira ao Cruzar o Meio Campo

Quando o jogador passa para o outro lado do campo para atacar o cesto oposto, a câmara roda suavemente 180° para que o jogador fique sempre de costas para a câmara e a olhar para o cesto correto.

Passo	Descrição
Deteção	Guardar lastSide = sign(playerPosition.z). Quando muda e != 0, houve cruzamento.
Target	cameraYawTarget += Math.PI (180°). Usar lerpAngle para evitar wrap incorreto.
Lerp	cameraYaw interpola suavemente ao longo de ~0.8s (fator 3 no exp).
Bloqueio	Flag isTurning = true durante a rotação. Ignora novos triggers até terminar.

Pseudocódigo:

```
const newSide = Math.sign(playerPosition.z); if (newSide !== 0 && newSide !== lastSide && !isTurning) { cameraYawTarget += Math.PI; isTurning = true; lastSide = newSide; }
cameraYaw = lerpAngle(cameraYaw, cameraYawTarget, 1 - Math.exp(-delta * 3)); if
(Math.abs(angleDiff(cameraYaw, cameraYawTarget)) < 0.01) isTurning = false;
```

2.2 — Sistema de Lançamento: Precisão Pura (Estilo 2K)

A barra passa a ser de **precisão/timing**, não de força. A velocidade é sempre a idealSpeed calculada para a distância — a bola chega sempre ao aro. O que varia é apenas o erro lateral (desvio horizontal).

Parâmetro	Sistema Atual	Sistema Novo
Velocidade	idealSpeed × (0.85 + power × 0.3)	idealSpeed sempre (constante)
Zona verde	Pequeno bônus ±1% velocidade	Erro horizontal = 0° (tiro perfeito)
Fora do verde	Velocidade errada + desvio	Só desvio lateral — bola sempre chega
Distância longa	Bola pode não chegar	Bola chega sempre ao aro
Label no HUD	'POWER'	'TIMING' ou 'PRECISÃO'

Alteração em shot.js — computeShotVelocity:

```
// REMOVER: speed = idealSpeed * (0.85 + power * 0.3); // SUBSTITUIR POR: speed =
idealSpeed; // velocidade ideal para a distância – sempre // Erro lateral mantém-se
(controla direção, não força): const center = (tuning.perfectMin + tuning.perfectMax) *
0.5; const missFactor = clamp(Math.abs(power - center) / center, 0, 1); horizontalError =
degToRad(tuning.maxHorizontalErrorDeg) * missFactor * (Math.random() < 0.5 ? -1 : 1);
```

2.3 — Novo Cenário Exterior: Campo de Parque

O ginásio fechado é substituído por um campo de parque ao ar livre, inspirado na foto de referência: superfície vermelha/verde, relva à volta, árvores no perímetro e postes de holofote nos cantos.

Superfície do campo:

Zona	Cor	Área
Zona de 3 pontos / paint	Vermelho (#C0392B)	Interior da linha de 3 pontos
Zona exterior do campo	Verde escuro (#1E8449)	Entre linha de 3 pontos e limite do campo
Relva envolvente	Verde natural (#27AE60)	Plano grande (60×60) à volta do campo
Linhas	Branco (#FFFFFF)	Sistema de textura atual mantém-se

Elementos do parque:

Elemento	Geometria	Qtd	Posição
Tronco de árvore	CylinderGeometry(0.25, 0.3, 10)	10–12	Perímetro da relva
Copa de árvore	SphereGeometry(1.5–2.0)	10–12	Topo de cada tronco
Bancos	BoxGeometry (existente adaptado)	4	Fora do campo, lados longos
Poste de holofote	CylinderGeometry(0.1, 0.1, 8)	8	Cantos do campo
Holofote (mesh)	BoxGeometry pequeno inclinado	1	Topo de cada poste
Piso relva	PlaneGeometry(60, 60)	1	y=0, centrado na cena

Elementos removidos do cenário atual:

Paredes (eastWall, westWall, northWall, southWall), teto (roof), bancadas (standA/B/C/D) e respetivos corpos físicos em world.js. Com o cenário aberto o clamp de câmara do ponto 1.5 passa a ser o único limite.

2.4 — Ciclo Dia/Noite com Transição Suave (Tecla T)

Ao pressionar T o jogo transita entre dia e noite (e vice-versa) ao longo de ~8 segundos. Todos os elementos visuais interpolam simultaneamente. O jogo continua jogável durante a transição.

Estado de Dia:

Elemento	Configuração
scene.background	Gradiente azul claro (#87CEEB → #E0F0FF)
Nuvens (grupos de esferas brancas)	Visíveis, opacity 0.85, movimento lento no eixo X
Sol (SphereGeometry)	Visível, y=40, z=-50, cor laranja/branca
DirectionalLight principal	Intensidade 0.9, cor branca/amarelada, sombras ativas
AmbientLight	Intensidade 0.5, cor azul claro
Lua + Estrelas + Holofotes	Invisíveis (opacity/intensity = 0)

Estado de Noite:

Elemento	Configuração
scene.background	Azul muito escuro uniforme (#0A0A1A)
Nuvens	Invisíveis (opacity 0)

Sol	Invisível, posição baixa (y=-10)
DirectionalLight principal	Intensidade 0 (apagada)
AmbientLight	Intensidade 0.15, cor azul frio (#1A1A3E)
Lua (SphereGeometry cinza clara)	Invisível, opacity 1, y=35, z=-45
DirectionalLight da lua	Intensidade 0.2, cor branco-azulada (#AAB8D4)
Estrelas (Points — 800 pontos)	Visíveis, opacity 0.9, esfera aleatória r=80
4× SpotLight (holofotes)	Intensidade 1.8, cone focado no campo, cor branca quente

Fases da transição (t: 0=dia → 1=noite):

Fase (t)	O que acontece
0.0 → 0.3	Sol começa a descer, céu fica laranja/rosado no horizonte (pôr do sol)
0.3 → 0.5	Céu escurece para roxo, sol desaparece, nuvens fazem fade out
0.5 → 0.7	Céu fica azul escuro, estrelas em fade in, lua começa a subir
0.7 → 1.0	Holofotes acendem em fade in, lua atinge posição final, céu negro completo

Controlador da transição em main.js:

```
const dayNight = { t: 0, transitioning: false, direction: 1, speed: 1/8 }; // Em
update(delta): if (input.wasKeyPressed('KeyT')) { dayNight.direction = dayNight.t < 0.5 ?
1 : -1; dayNight.transitioning = true; } if (dayNight.transitioning) { dayNight.t =
clamp(dayNight.t + dayNight.direction * dayNight.speed * delta, 0, 1);
applyDayNightState(dayNight.t); if (dayNight.t === 0 || dayNight.t === 1)
dayNight.transitioning = false; }
```

PATCH 3

Animações, Câmera 1ª Pessoa e Visual

Animações procedurais, aro maior, céu realista e câmara imersiva

3.1 — Aro Maior (Exagerado para Jogabilidade)

O aro atual tem escala real mas fica difícil de acertar, mesmo no verde. O raio será aumentado para ~0.38 o que visualmente ainda parece credível mas é muito mais permissivo. A deteção em `scoring.js` usa as constantes e ajusta automaticamente.

Constante	Patch 1	Patch 3	Nota
COURT.rimRadius	0.34	0.40	Pode ajustar se necessário após teste
COURT.rimTubeRadius	0.03	0.035	Tubo mais visível com aro maior
PHYSICS.rimSegmentRadius	0.03	0.035	Consistente com o tubo
effectiveRadius em <code>scoring.js</code>	$\text{rimRadius} - \text{ballRadius} \times 0.85$	0.35	Ajusta automaticamente

3.2 — Animações Procedurais do Jogador

As animações são feitas matematicamente com funções seno/cosseno em cada frame, sem ficheiros externos. O problema anterior foi a falta de pivots corretos nas pernas. Serão implementadas 4 animações distintas com transições suaves entre elas.

Correção dos Pivots das Pernas:

As pernas atuais são meshes diretos no grupo — rodam em torno do centro da coxa. É necessário criar Groups intermédios com o pivot na anca, igual ao sistema dos braços.

Estrutura correta em `player.js`:

```
// ANTES (errado – sem pivot): const leftLeg = new THREE.Mesh(CylinderGeometry(0.1, 0.09, 0.88, 14), socks); leftLeg.position.set(-0.16, 0.2, 0); // DEPOIS (correto – com pivot na anca): const leftLegPivot = new THREE.Group(); leftLegPivot.position.set(-0.16, 0.42, 0); // posição da anca const leftLeg = new THREE.Mesh(CylinderGeometry(0.1, 0.09, 0.88, 14), socks); leftLeg.position.y = -0.44; // metade da altura – pende para baixo leftLegPivot.add(leftLeg); group.add(leftLegPivot);
```

As 4 Animações:

Animação	Trigger	O que anima	Fórmula base
Idle Dribble	<code>hasBall && !isMoving</code>	Braço direito sobe/desce, torso inclina ligeiramente	$\sin(\text{time} \times 1.5)$ e $\sin(\text{time} \times 1.5)$
Walk Dribble	<code>hasBall && isMoving</code>	Pernas balançam alternadamente, braços balançam para os lados	$\sin(\text{time} \times 2.0)$ e $\sin(\text{time} \times 2.0)$
Shoot	<code>shotCharge shoot</code>	Braços e pernas levam-se progressivamente para cima	$\sin(\text{time} \times 1.5)$ e $\sin(\text{time} \times 1.5)$
Celebrate	<code>celebrationTimer > 0</code>	Braços ao alto, torso inclina para trás, cabeça para trás, salto com sin(p)	$\sin(\text{time} \times 1.5)$ e $\sin(\text{time} \times 1.5)$

Sistema de blending entre animações:

As animações não mudam abruptamente — cada parâmetro interpola com lerp em direção ao valor alvo da animação ativa. Isto evita saltos visuais ao mudar de estado (ex: parar de andar, começar a lançar).

Pseudocódigo do sistema de animação em `player.js`:

```
// Chamado em cada frame com o estado atual function updateAnimations(delta, state,
animTime) { let targetLeftLeg = 0, targetRightLeg = 0; let targetLeftArm = 0,
targetRightArm = 0; if (state === 'WALK_DRIBBLE' || state === 'MOVING') { targetLeftLeg =
Math.sin(animTime * 10) * 0.45; targetRightLeg = -Math.sin(animTime * 10) * 0.45;
targetLeftArm = -Math.sin(animTime * 10) * 0.3; targetRightArm = Math.sin(animTime * 10)
* 0.3; } else if (state === 'IDLE_DRIBBLE') { targetRightArm = Math.sin(animTime * 8) *
0.4 - 0.3; } const alpha = 1 - Math.exp(-delta * 8); // blending suave
leftLegPivot.rotation.x = lerp(leftLegPivot.rotation.x, targetLeftLeg, alpha);
rightLegPivot.rotation.x = lerp(rightLegPivot.rotation.x, targetRightLeg, alpha); // ...
idem para braços }
```

3.3 — Céu Procedural Mais Realista

O fundo atual é uma cor sólida ou gradiente simples. Será criado um céu procedural com ShaderMaterial que simula dispersão atmosférica — gradiente suave do zenith para o horizonte, mais realista e sem ficheiros externos.

Elemento	Técnica	Detalhe
Céu de dia	SphereGeometry grande (r=200) de ShaderMaterial com gradientes	Zenith azul escuro (#1565C0) → horizonte azul claro
Nuvens	Grupos de 3–5 SphereGeometrys com ShaderMaterial transparente	Entre y=0.85, y entre 18 e 28, movimento lento
Sol	SphereGeometry(1.5) + MeshBasicMaterial própria posição	Base no centro da posição y=40 z=-50, PointLight laranja à vo
Céu de noite	Mesmo SphereGeometry, com gradientes	Zenith #0A26210 → horizonte #0A0A2A
Estrelas	Points com BufferGeometry, 8000 pontos	8000 posições aleatórias em esfera r=150, PointsMaterial size=0.3
Lua	SphereGeometry(2.5) + MeshBasicMaterial própria posição	Posição y=35 z=70, sem necessidade de sombras

Todo o código do céu será isolado num ficheiro `src/world/sky.js` para não poluir o `court.js`. Exporta `createSky()` que devolve os meshes e uma função `update(delta)` para mover as nuvens.

3.4 — Câmara em Primeira Pessoa com Lock-On ao Cesto

Ao pressionar P (ou botão no HUD), o jogo alterna entre 3ª pessoa e 1ª pessoa. Em 1ª pessoa o rato controla o look livremente. Ao iniciar um lançamento, o jogador e a câmara rodam automaticamente para encarar o cesto — lock-on suave.

Aspeto	3ª Pessoa	1ª Pessoa
Posição da câmara	Atrás e acima do jogador (offset)	Na cabeça do jogador (head.position)
Controlo da câmara	Automático — segue o jogador	Rato — Pointer Lock API
Personagem visível	Sim	Não (câmara está dentro do modelo)
WASD	Relativo à câmara (estilo 2K)	Relativo à direção que estás a olhar
Lançamento	Para o cesto mais próximo	Lock-on suave ao cesto → depois lança
HUD	Igual	Igual — barra de timing mantém-se

Fluxo do Lock-On ao Cesto em 1ª Pessoa:

Fase	Duração	O que acontece
------	---------	----------------

1. Trigger	Instantâneo	Jogador clica — jogo calcula ângulo para o cesto mais próximo
2. Lock-On	~0.35 segundos	playerYaw e cameraYaw interpolam para encarar o cesto. Rato bloqueado.
3. Charge	Enquanto segurar	Bola de timing aparece. Câmara fica fixa a olhar para o cesto.
4. Release	Instantâneo	Bola lançada. Rato volta a ser livre após 0.6s.

Implementação do Pointer Lock e look em camera.js:

```
// Ativar Pointer Lock ao entrar em 1ª pessoa
renderer.domElement.requestPointerLock();
// No handler de mousemove (só em 1ª pessoa e sem lock-on ativo):
document.addEventListener('mousemove', (e) => {
  if (!isFirstPerson || isLockingOn) return;
  fpYaw -= e.movementX * 0.002;
  fpPitch -= e.movementY * 0.002;
  fpPitch = clamp(fpPitch, -Math.PI/3, Math.PI/3); // limite vertical
}); // Posição da câmara em 1ª pessoa:
const headPos = new THREE.Vector3(0, 1.85, 0.05); // offset da cabeça
camera.position.add(playerPosition);
camera.position.copy(headPos);
camera.rotation.set(fpPitch, fpYaw, 0, 'YXZ');
```

Ordem de Implementação Global Recomendada

A ordem abaixo minimiza retrabalho — cada melhoria constrói sobre a anterior e pode ser testada de forma isolada antes de avançar.

Ordem	Melhoria	Razão
1º	1.1 — A/D invertidos	Fix de 1 linha, valida controlos base imediatamente
2º	1.3 — Rede invertida	Visual simples, confirma que court.js está ok
3º	1.4 — Escala bola/cesto	Ajuste de constantes, testar scoring
4º	3.1 — Aro maior	Continuação da escala, testar acertos
5º	2.2 — Precisão pura (shot.js)	Isolado em shot.js, testar lançamentos de todas as distâncias
6º	1.6 — Brilho e câmara	Ajusta perspetiva antes de mudar movimento
7º	1.5 — Câmara no campo	Depende do novo offset da câmara do passo anterior
8º	1.2 — Movimento 2K	Sistema de input — testar bem antes de avançar
9º	2.1 — Câmara vira no meio campo	Depende do sistema de movimento 2K
10º	2.3 — Cenário exterior	Maior refactor — fazer depois da lógica estar estável
11º	3.3 — Céu procedural	Depende do cenário exterior sem teto
12º	2.4 — Ciclo dia/noite	Depende dos postes/holofotes do cenário exterior
13º	3.2 — Animações procedurais	Isolado em player.js, pode ser feito a qualquer altura
14º	3.4 — Câmara 1ª pessoa	Deixar para o fim — depende de tudo estar estável

Ferramentas de IA integradas no desenvolvimento e apoio à implementação: OpenAI ChatGPT · Anthropic Claude